

Manual Multilayer Perceptron Classification with Backpropagation

Technical Analysis of a Python Implementation

This report describes the computational, mathematical, and software structure of the supplied Python source code. It covers the neural-network pipeline, data and weight handling, learning procedure, and implementation constraints that can be established directly from the source.

Contents

1	Introduction	2
2	Project Overview	3
3	Technical Foundations	3
3.1	Feed-Forward Multilayer Perceptron	3
3.2	Sigmoid Activation	4
3.3	Backpropagation	4
4	Data and Input Processing	4
5	Weight Representation and Initialization Parsing	5
6	System Architecture and Code Organization	5
7	Detailed Methodology	5
7.1	Forward Propagation	5
7.2	Output Error and Delta	6
7.3	Hidden-Layer Backpropagation	7
7.4	Parameter Updates	7
8	Optimization and Learning Procedure	7
9	Testing and Prediction Rule	8
10	Mathematical Formulation of the Implemented Learner	8
11	Implementation Details	9
11.1	Library Use	9
11.2	Data Structure Boundaries	9
11.3	In-Place Mutation	9
11.4	Numerical Stability	9
12	Execution Workflow	9
13	Design Decisions and Rationale	10

14 Computational Considerations	10
15 Reproducibility and Reliability	11
16 Limitations and Technical Considerations	11
16.1 Target Extraction Defect	11
16.2 Undefined Evaluation Function	11
16.3 Synthetic Test Labels	12
16.4 Repeated Debug Output	12
16.5 Incomplete Validation and Configuration	12
16.6 No Explicit Loss Tracking or Convergence Criterion	12
17 Overall Technical Assessment	12
18 Conclusion	13

Source Boundary and Evidence Policy

This report stays within what can be established from the supplied Python program. The file contains 118 physical source lines and uses `pandas` and `numpy`. It also expects an external Excel workbook named `Computer_Assignment_4_Data.xls`. That workbook was not provided, so the report does not state dataset dimensions, initial-weight values, training accuracy, or runtime measurements that cannot be determined from the source alone.

The discussion below separates three kinds of evidence: behavior that is explicit in the code, standard mathematical interpretations of that behavior, and reasonable inferences from names or structure. When the implementation contains an inconsistency or a missing component, it is reported rather than silently corrected. This keeps the mathematical description separate from what the program actually does.

Abstract

The supplied program implements a small multilayer perceptron (MLP) classifier with NumPy, a sigmoid activation function, and manually coded backpropagation. It reads training data, test data, and initial network weights from separate Excel sheets, extracts the input-to-hidden and hidden-to-output weight matrices, performs a forward pass through one hidden layer, computes output and hidden-layer error signals, and updates the parameters one sample at a time. During inference, the two output activations are converted to class labels with an `argmax` rule.

The script is intentionally low level: it does not use a neural-network framework, automatic differentiation, or a pre-built classifier. As a result, the main computations are visible in the code, which makes the implementation easy to inspect. The source also has several correctness and robustness limitations. The training target slice is inconsistent with the two-column `y_train` object, `evaluate_results` is not defined in the supplied file, and the test labels are inferred from an unstated row-order assumption. In addition, `forward_pass` prints the hidden weight matrix on every call, producing unnecessary console output during training.

1 Introduction

The project implements supervised binary classification with a feed-forward neural network. Its main task is to map two numerical features to a two-dimensional output representing two classes. Instead of relying on a machine-learning framework for training, the program explicitly performs the core MLP operations: affine transformations, nonlinear activation, error propagation, and parameter updates.

This implementation makes the mechanics of neural-network learning easy to inspect. The code shows how bias terms are handled, how the sigmoid derivative is used, how the output error is propagated to the hidden layer, and how the weight updates are formed with outer products. Its main value as an educational implementation is that these operations are explicit rather than hidden behind a high-level API.

This report focuses on the implementation itself. It follows the pipeline from workbook

loading through preprocessing, weight initialization, forward propagation, backpropagation, optimization, testing, and the attempted evaluation step. It also records assumptions and execution-blocking defects that are visible in the source. Because the workbook was not supplied, no sample values or empirical performance are inferred.

2 Project Overview

The program can be viewed as five stages. It first loads the Excel workbook and reads the training set, test set, and initial-weight specification. It then parses the weight specification into the two layer-specific matrices. Next, it defines the sigmoid function and forward pass. Training iterates over epochs and samples, computes the error signals, and updates both weight matrices. Finally, the trained network is applied to the test set and the predictions are passed to the evaluation step.

The top-level execution is linear. There is no model class, configuration class, or separate training script. The main network state is stored in two NumPy arrays, `weights_input_hidden` and `weights_hidden_output`. The input data remain in pandas objects, while the trainable parameters are converted to NumPy arrays before training.

The design can be summarized as Excel workbook $\rightarrow$ parsed data/weights $\rightarrow$ forward pass $\rightarrow$ backpropagation $\rightarrow$ updated weights $\rightarrow$ test predictions $\rightarrow$ evaluation call. The final step is incomplete because the supplied source references, but does not define, the evaluation function.

3 Technical Foundations

3.1 Feed-Forward Multilayer Perceptron

The implemented model is a feed-forward neural network with one hidden layer. Let the input vector be $\mathbf{x} \in \mathbb{R}^d$, the hidden-layer width be h , and the output dimension be two. The hidden layer applies an affine transformation followed by a sigmoid activation, and the output layer uses the same pattern.

For an input vector $\mathbf{x}$, the hidden pre-activation is

$$\mathbf{z}^{(1)} = (\tilde{\mathbf{x}})^\top \mathbf{W}^{(1)}, \quad (1)$$

where $\tilde{\mathbf{x}} = [x_1, \dots, x_d, 1]^\top$ is the augmented input vector containing a bias coordinate, and the corresponding final row of $\mathbf{W}^{(1)}$ stores the hidden-layer bias parameters. The hidden activation is

$$\mathbf{h} = \sigma(\mathbf{z}^{(1)}). \quad (2)$$

The output layer similarly forms

$$\mathbf{z}^{(2)} = (\tilde{\mathbf{h}})^\top \mathbf{W}^{(2)}, \quad (3)$$

where $\tilde{\mathbf{h}} = [h_1, \dots, h_h, 1]^\top$. The final output is

$$\hat{\mathbf{y}} = \sigma(\mathbf{z}^{(2)}). \quad (4)$$

The code implements this structure directly instead of using a general-purpose neural-network object.

3.2 Sigmoid Activation

The activation function is the logistic sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (5)$$

Its derivative can be expressed compactly as

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (6)$$

The training implementation uses the activation values themselves to compute this derivative, avoiding a second exponential evaluation.

3.3 Backpropagation

The training routine follows the usual local-error backpropagation pattern for a sigmoid-output network with a squared-error-style delta rule. The output-layer error signal is

$$\mathbf{e}^{(2)} = \mathbf{t} - \hat{\mathbf{y}}, \quad (7)$$

and the output delta is

$$\boldsymbol{\delta}^{(2)} = \mathbf{e}^{(2)} \odot \hat{\mathbf{y}} \odot (\mathbf{1} - \hat{\mathbf{y}}), \quad (8)$$

where $\odot$ denotes element-wise multiplication. The hidden error signal is then propagated through the transpose of the non-bias part of the output weight matrix:

$$\mathbf{e}^{(1)} = \boldsymbol{\delta}^{(2)} \left(\mathbf{W}_{\text{nb}}^{(2)} \right)^T, \quad (9)$$

followed by

$$\boldsymbol{\delta}^{(1)} = \mathbf{e}^{(1)} \odot \mathbf{h} \odot (\mathbf{1} - \mathbf{h}). \quad (10)$$

This follows the chain-rule structure for a one-hidden-layer network. The code updates the parameters immediately after each sample, so the training is sample-wise rather than full-batch.

4 Data and Input Processing

The program expects an Excel workbook named `Computer_Assignment_4_Data.xls` with at least three sheets: `Training Set`, `Testing Set`, and `Initial Weights`. The training sheet is read with the columns `Class #`, `x1`, `x2`, `Target1`, and `Target2`; the testing sheet uses `Class #`, `x1`, and `x2`. The weight sheet is read without a header because it contains both section markers and numerical values.

The source handles `Class #` separately from `x1` and `x2`. Although `X_train` initially contains all three columns, the training loop uses `inputs.iloc[i, 1:]`, so the network receives only x_1

and x_2 . The `Class #` field is not used as an input feature. Testing follows the same convention through `test_set.iloc[i, 1:]`.

The target frame contains only `Target1` and `Target2`. However, the training loop later accesses columns 3 through 4 of this two-column frame. With pandas integer-location slicing, `targets.iloc[i, 3:5]` returns an empty slice instead of the two target values. As written, the program therefore cannot complete its first training sample after the workbook is loaded.

5 Weight Representation and Initialization Parsing

The initial-weight sheet uses text markers to delimit the two weight blocks. The program locates `Input to Hidden Weights` and starts the first block on the next numeric row. It then looks for `Hidden to Output Weights`; when that marker is present, its row ends the first block. Otherwise, the code attempts to end the block immediately before the `Learning Rate` marker.

The output-layer block is parsed only when the `Hidden to Output Weights` marker is present. The fallback therefore handles one workbook variation but has a clear limitation: without that marker, `hidden_output_weights` is an empty `DataFrame`. The training function then converts it to a NumPy array and uses it in matrix multiplication, so the resulting model is not trainable with such a workbook.

The internal representation stores bias weights in the last row of each weight matrix. Both the forward pass and the update equations use this convention. For the hidden layer, the feature vector is multiplied by the non-bias rows and the final row is added as the bias term. The output layer uses the same arrangement with the hidden activations.

6 System Architecture and Code Organization

The code fits in a single source file and defines four functions. `sigmoid` provides the activation function. `forward_pass` performs the two affine transformations and sigmoid activations. `train_mlp` contains the training loop and parameter updates. `test_mlp` performs inference and converts the two output activations to integer class labels.

The rest of the file is top-level orchestration. It loads the workbook, extracts the data, parses the weight matrices, constructs the training and test objects, runs training and testing, creates an assumed true-label vector, and calls `evaluate_results`. There is no `main()` function, so importing the module would also execute the file I/O and training code.

7 Detailed Methodology

7.1 Forward Propagation

The implementation receives a two-element feature vector and handles the bias separately. If the hidden weight matrix has shape $(d + 1) \times h$, the code computes

$$\mathbf{z}^{(1)} = \mathbf{x}^T \mathbf{W}_{\text{nb}}^{(1)} + \mathbf{b}^{(1)}, \quad (11)$$

Component	Source region	Role
Excel loading	top-level setup	Reads training, testing, and initial-weight sheets.
Weight parsing	top-level setup	Locates section markers and extracts weight blocks.
<code>sigmoid</code>	function definition	Computes the logistic activation element-wise.
<code>forward_pass</code>	function definition	Computes hidden and output activations, including bias handling.
<code>train_mlp</code>	function definition	Runs sample-wise epochs, backpropagation, and weight updates.
<code>test_mlp</code>	function definition	Generates class predictions from output activations.
Evaluation call	final top-level block	Invokes an undefined <code>evaluate_results</code> function.

Table 1: Functional decomposition of the supplied source code.

where $\mathbf{W}_{\text{nb}}^{(1)}$ denotes all but the final row of the matrix and $\mathbf{b}^{(1)}$ is the final row. The hidden activations are then obtained through the sigmoid function.

The output stage applies the same construction:

$$\mathbf{z}^{(2)} = \mathbf{h}^\top \mathbf{W}_{\text{nb}}^{(2)} + \mathbf{b}^{(2)}, \quad (12)$$

followed by $\hat{\mathbf{y}} = \sigma(\mathbf{z}^{(2)})$.

One implementation detail is especially costly: `forward_pass` prints the entire input-to-hidden weight matrix before returning. The function is called for every training sample and every test sample, so this print statement runs inside the main computation loop. With 500 epochs, the amount of output grows with the number of training samples times 500 and can become very large.

7.2 Output Error and Delta

The training code forms a target-minus-prediction residual,

$$\mathbf{e}^{(2)} = \mathbf{t} - \hat{\mathbf{y}}, \quad (13)$$

and multiplies it element-wise by the sigmoid derivative. This yields

$$\boldsymbol{\delta}^{(2)} = (\mathbf{t} - \hat{\mathbf{y}}) \odot \hat{\mathbf{y}} \odot (1 - \hat{\mathbf{y}}). \quad (14)$$

This sign convention matches the code’s choice to add the update to the current weights rather than subtract a gradient written with the opposite error sign.

The code does not compute a scalar loss value. There is therefore no loss history, convergence plot, or loss-based stopping criterion. The optimizer always runs for the fixed number of epochs supplied by the caller.

7.3 Hidden-Layer Backpropagation

The hidden-layer error is computed using the non-bias output weights:

$$\mathbf{e}^{(1)} = \boldsymbol{\delta}^{(2)} (\mathbf{W}_{\text{nb}}^{(2)})^\top. \quad (15)$$

The hidden delta then follows from the sigmoid derivative:

$$\boldsymbol{\delta}^{(1)} = \mathbf{e}^{(1)} \odot \mathbf{h} \odot (1 - \mathbf{h}). \quad (16)$$

This excludes the output-layer bias weights from the hidden-error calculation. That is appropriate because the bias is not a hidden unit and is not part of the chain-rule path back to the hidden activations.

7.4 Parameter Updates

The hidden-to-output weights are updated with the outer product of hidden activations and output deltas:

$$\mathbf{W}_{\text{nb}}^{(2)} \leftarrow \mathbf{W}_{\text{nb}}^{(2)} + \eta \mathbf{h} (\boldsymbol{\delta}^{(2)})^\top, \quad (17)$$

with the output bias updated as

$$\mathbf{b}^{(2)} \leftarrow \mathbf{b}^{(2)} + \eta \boldsymbol{\delta}^{(2)}. \quad (18)$$

Analogously, the input-to-hidden matrix is updated as

$$\mathbf{W}_{\text{nb}}^{(1)} \leftarrow \mathbf{W}_{\text{nb}}^{(1)} + \eta \mathbf{x} (\boldsymbol{\delta}^{(1)})^\top, \quad (19)$$

and the hidden bias becomes

$$\mathbf{b}^{(1)} \leftarrow \mathbf{b}^{(1)} + \eta \boldsymbol{\delta}^{(1)}. \quad (20)$$

The use of `np.outer` makes this structure explicit: each connection weight is adjusted according to the presynaptic activation and postsynaptic error signal.

8 Optimization and Learning Procedure

The call to `train_mlp` uses a learning rate of $\eta = 0.2$ and 500 epochs, giving the training a fixed budget. The source does not use minibatches, momentum, an adaptive learning-rate schedule, weight decay, gradient clipping, or early stopping.

The nested loops implement the sequence

1. select one training sample;
2. compute hidden and output activations;
3. compute the output delta;
4. propagate the delta to the hidden layer;
5. update output weights and bias;

6. update hidden weights and bias;
7. move directly to the next sample.

Because the weights are updated after each sample, later samples in an epoch use parameters that already reflect earlier samples. This is the defining behavior of online or stochastic gradient-style training.

The source does not shuffle samples between epochs, so the row order in the training sheet determines the update order in every epoch. If the data are ordered by class, the resulting optimization path can differ from that of a shuffled implementation even with the same samples and hyperparameters.

9 Testing and Prediction Rule

After training, `test_mlp` computes a two-dimensional output for each test sample. The predicted class is selected by

$$\hat{c} = 1 + \arg \max_{j \in \{1,2\}} \hat{y}_j. \quad (21)$$

The addition of one converts NumPy's zero-based index to class labels 1 and 2.

This decision rule is consistent with a two-output representation when the targets use two components and the larger activation selects the class. However, the source does not verify that the targets are one-hot encoded or that the two target columns correspond exactly to classes 1 and 2. That relationship is inferred from `Target1`, `Target2`, and the prediction rule.

The test-label construction is more fragile. The program creates `[1] * (n // 2) + [2] * (n // 2)`. This assumes that the number of test samples is even and that the first half of the rows belongs to class 1 while the second half belongs to class 2. The workbook, not the Python file, would determine whether that ordering is correct. The code therefore does not extract the actual test labels from the test sheet.

10 Mathematical Formulation of the Implemented Learner

Let $\mathbf{x} \in \mathbb{R}^d$ denote the feature vector, h the hidden width, and $K = 2$ the output dimension. With explicit bias terms, the network can be expressed compactly as

$$\mathbf{z}^{(1)} = \mathbf{x}^\top \mathbf{W}_{\text{nb}}^{(1)} + \mathbf{b}^{(1)}, \quad (22)$$

$$\mathbf{h} = \sigma(\mathbf{z}^{(1)}), \quad (23)$$

$$\mathbf{z}^{(2)} = \mathbf{h}^\top \mathbf{W}_{\text{nb}}^{(2)} + \mathbf{b}^{(2)}, \quad (24)$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{z}^{(2)}). \quad (25)$$

The code's delta rule can be written as

$$\boldsymbol{\delta}^{(2)} = (\mathbf{t} - \hat{\mathbf{y}}) \odot \hat{\mathbf{y}} \odot (\mathbf{1} - \hat{\mathbf{y}}), \quad (26)$$

$$\boldsymbol{\delta}^{(1)} = \left(\boldsymbol{\delta}^{(2)} (\mathbf{W}_{\text{nb}}^{(2)})^\top \right) \odot \mathbf{h} \odot (\mathbf{1} - \mathbf{h}). \quad (27)$$

For the non-bias weights, a single-sample update is therefore

$$\mathbf{W}_{\text{nb}}^{(2)} \leftarrow \mathbf{W}_{\text{nb}}^{(2)} + \eta \mathbf{h}(\boldsymbol{\delta}^{(2)})^\top, \quad (28)$$

$$\mathbf{W}_{\text{nb}}^{(1)} \leftarrow \mathbf{W}_{\text{nb}}^{(1)} + \eta \mathbf{x}(\boldsymbol{\delta}^{(1)})^\top. \quad (29)$$

These equations describe the mathematical intent of the update sequence. They do not show that the script defines a formal scalar objective, because no such objective is explicitly defined or recorded in the source.

11 Implementation Details

11.1 Library Use

Only two third-party Python libraries are imported: `pandas` and `numpy`. `Pandas` handles Excel I/O and tabular slicing, while `NumPy` provides arrays, dot products, concatenation, element-wise operations, exponentiation, and outer products. The source has no dependency on `TensorFlow`, `PyTorch`, `scikit-learn`, or automatic differentiation.

11.2 Data Structure Boundaries

The implementation has a clear representation boundary. Spreadsheet-derived data remain `pandas DataFrames`, while the trainable weight matrices are converted to `NumPy` arrays at the start of `train_mlp`. This is a practical division for a small script: `pandas` is convenient for tabular indexing, and `NumPy` is suited to the repeated numerical operations.

11.3 In-Place Mutation

Both weight matrices are modified in place during training, so the returned arrays contain the final parameter values after all epochs. The training routine is therefore stateful even though it is written as a function with explicit array arguments. The input `DataFrames` are otherwise left unchanged apart from the earlier column-name normalization applied to `X_train`.

11.4 Numerical Stability

The sigmoid implementation uses the direct expression $1/(1 + e^{-x})$. This is adequate for moderate values, but it can overflow in the exponential for large negative inputs and can saturate for large-magnitude inputs. The source includes no clipping, stabilized sigmoid formulation, or warning handling. The practical impact cannot be assessed without the initial weights and feature values from the workbook.

12 Execution Workflow

At source level, the execution sequence is straightforward. The workbook is opened first. The training and testing sheets are read with fixed column names, and the initial-weight sheet

is loaded without headers. The code then searches for the weight-block markers, defines the model functions, and constructs `X_train`, `y_train`, and `X_test`.

Next, `train_mlp` is called with the parsed weights, a learning rate of 0.2, and 500 epochs. In the intended execution, each sample goes through a forward and backward pass followed by an immediate update. `test_mlp` then processes the test samples, a synthetic true-label vector is created using the equal-halves assumption, and `evaluate_results` is called.

The final step cannot execute because the supplied source neither defines nor imports `evaluate_results`. More importantly, execution is expected to fail earlier during the first training sample because of the target-slice inconsistency described above. The sequence should therefore be read as the intended workflow, not as evidence of a successful end-to-end run.

13 Design Decisions and Rationale

The main design choice is to implement the neural-network algorithm explicitly. This exposes details that high-level libraries normally hide, including bias handling, matrix shapes, the chain rule, activation derivatives, and sample-wise updates. For an educational assignment, this makes the learning process easy to inspect.

The source also uses a spreadsheet as the interface between data preparation and model execution. That is practical when an assignment distributes data and initial weights in workbook sheets, but it makes the pipeline depend on an external file with exact sheet names and marker strings.

The separate functions for forward propagation, training, and testing provide useful conceptual boundaries even without a formal model class. In particular, `train_mlp` calls `forward_pass` instead of duplicating the activation calculations, so that part of the computation remains centralized.

Some choices are visible in the code but are not explained, so their rationale should not be inferred. The source uses 500 epochs and a learning rate of 0.2, but it does not say why those values were chosen. The test-label construction likewise suggests an expected row order without documenting or checking it.

14 Computational Considerations

Let N be the number of training samples, d the number of input features, h the number of hidden units, and $K = 2$ the number of outputs. A forward and backward pass is dominated by matrix-vector products and element-wise operations. For one sample, the leading arithmetic is approximately

$$O(dh + hK), \tag{30}$$

with additional lower-order element-wise work. Over E epochs and N samples, the training cost is approximately

$$O(EN(dh + hK)). \tag{31}$$

The source uses explicit Python loops over epochs and samples. The numerical operations inside each iteration are handled by NumPy, but the per-sample control flow remains in

Python. This is reasonable for a small educational dataset, though it is less scalable than vectorized minibatch training.

Memory use is modest for a small MLP because the persistent model state consists mainly of the two weight matrices. There is no replay buffer, optimizer state, or stored activation history across the dataset. The main practical issue is therefore not model size but the large amount of console output produced by `forward_pass`.

15 Reproducibility and Reliability

The program does not generate random numbers, so the operations visible in the source do not require a random seed. This improves reproducibility because the initial weights are read from the workbook rather than sampled inside the script.

Reproducibility still depends on using the same workbook contents, sheet names, row ordering, marker labels, numeric types, and initial-weight layout. The learning rate and epoch count are also hard-coded at the training call site. The supplied source contains no dependency lock file, environment specification, or library-version checks.

Reliability is limited by the lack of input validation. The program does not check that the required sheets exist, that the marker strings occur exactly once, that the parsed matrices have compatible shapes, that the target has two elements, or that the test set has an even number of rows. A more robust implementation would check these assumptions and raise informative errors.

16 Limitations and Technical Considerations

16.1 Target Extraction Defect

The clearest correctness defect is the mismatch between how `y_train` is constructed and how the training loop slices it. `y_train` has exactly two columns, but the later slice requests positions 3 through 4. The resulting target array is empty. The line

```
target = targets.iloc[i, 3:5].to_numpy()
```

does not extract the intended target pair. Based on the supplied source, this is a blocking training error, not a stylistic issue.

16.2 Undefined Evaluation Function

The final statement calls `evaluate_results(predictions, true_labels)`, but that function is neither defined nor imported. The supplied file therefore cannot complete the evaluation stage on its own.

16.3 Synthetic Test Labels

The construction of `true_labels` does not read labels from the test data. It assumes an even number of test observations and a fixed order in which half are class 1 followed by class 2. That may match the workbook convention, but the source does not verify it.

16.4 Repeated Debug Output

The print statement inside `forward_pass` runs for every sample in every epoch. It can therefore produce a large amount of output and adds I/O to the main training loop.

16.5 Incomplete Validation and Configuration

The parser depends on exact marker text and positional conventions. It performs no explicit shape validation or error reporting for malformed inputs. The script also assumes that the external `.xls` file is available in the current working directory and can be read by the installed pandas Excel engine.

16.6 No Explicit Loss Tracking or Convergence Criterion

The source does not compute or store a scalar loss over time, and the epoch count is fixed. The code therefore provides no direct way to determine whether optimization converged, oscillated, or stopped before reaching an adequate solution.

17 Overall Technical Assessment

The implementation shows a clear understanding of the core mechanics of a small feed-forward neural network. Its main strength is transparency: the sigmoid activation, forward propagation, backpropagation, bias handling, and parameter updates are visible rather than hidden behind a framework. The use of outer products for the connection updates and the exclusion of bias rows from hidden-error propagation are also technically appropriate.

The software structure is also coherent for a compact assignment. The forward pass, training loop, and testing routine are separated into distinct functions, with pandas handling the workbook and NumPy handling numerical computation. Because there is no framework-level training behavior, the script is straightforward to inspect mathematically.

The supplied source should not be presented as a verified, fully executable implementation without qualification. The target-extraction bug prevents correct training, the evaluation function is missing, and the true-label construction depends on an unverified ordering assumption. These are execution and evaluation issues, not matters of style. From a reproducibility perspective, the code illustrates that a sound algorithmic idea still depends on consistent data interfaces, explicit validation, and an end-to-end execution check.

18 Conclusion

The project is a direct NumPy implementation of a two-output multilayer perceptron with sigmoid activations and sample-wise backpropagation. Its computation consists of affine transformations, nonlinear activations, error signals, and outer-product weight updates. Bias parameters are represented explicitly as the final row of each weight matrix.

The implementation is useful as an example of low-level neural-network programming because it exposes the mechanics of learning instead of relying on automatic differentiation or a high-level training API. Its structure is compact and readable, and the numerical operations closely follow the standard chain-rule formulation of backpropagation.

The source audit also sets clear limits on what can be claimed. The external workbook is needed to determine concrete dimensions and values, and the supplied script contains defects that prevent an unqualified claim of successful end-to-end execution. The technical description should therefore distinguish the intended mathematical architecture from the actual execution and explicitly note the target-indexing error, missing evaluation routine, label-order assumption, repeated debug output, and other reliability constraints.

Source Traceability Map

For practical review, the main implementation regions are workbook loading and data extraction (approximately lines 4–14), weight-block parsing (approximately lines 16–36), the sigmoid activation and forward pass (approximately lines 40–52), training and backpropagation (approximately lines 56–85), testing and prediction (approximately lines 87–96), and top-level orchestration and evaluation (approximately lines 98–118). These regions link the report’s discussion directly to the supplied source.